

Explainability Report

Ezitech Engineering Framework — Case Study AI-005

Internship Performance Prediction & Risk Analytics Platform

Prepared by: Sadaf Riaz & Saira Ejaz

1. Why Explainability Matters Here

A prediction like "68% dropout risk" is not, on its own, actionable for a mentor — it doesn't say what to do about it. The case study requires an Explainable AI Module for exactly this reason: every prediction served by this platform is paired with a SHAP (SHapley Additive exPlanations) explanation identifying which specific features drove that individual prediction, so a mentor can see, for a given intern, whether the risk is coming from declining attendance, weak code reviews, or something else — and act on the actual cause rather than the score alone.

2. Method: SHAP TreeExplainer

All four trained models are tree-based or linear, so `shap.TreeExplainer` is used throughout (`app/ml/predict.py`, `_shap_explanation()`). For each prediction request, the explainer computes a SHAP value for every input feature — the marginal contribution of that feature, for that specific intern, toward the model's output — and the top 3 contributing features (by absolute value) are returned alongside the prediction itself as `top_factors`, each with a feature name and a signed contribution value.

This is a per-prediction, local explanation, not a static, global feature-importance ranking computed once at training time. The distinction is what makes it useful in practice: two interns with the same dropout-risk score can have that score driven by entirely different underlying factors, and only a per-prediction explanation reveals that.

3. Handling Multi-Class Explanations

Performance Trend is a 3-class model (declining / stable / improving), and SHAP returns a separate set of contribution values per class rather than one single set. An early implementation assumed a single-array (binary-style) shape and raised a `ValueError` on 3-class predictions. This was corrected by explicitly selecting the SHAP values for the class that was actually predicted (`class_index`), and by handling both output shapes that different SHAP/XGBoost versions can return — a list of per-class arrays, or a single 3D array indexed as `[sample, feature, class]` — rather than assuming one fixed shape.

Explanations also degrade safely: if SHAP computation fails for any reason, the API returns a note explaining that the explanation is unavailable rather than failing the whole prediction request.

4. What Each Model's Explanation Contains

Endpoint	Explanation Contents
<code>/predict/dropout-risk</code>	<code>top_factors</code> (3 features), <code>flagged_as_at_risk</code> (bool), <code>decision_threshold</code>
<code>/predict/performance-trend</code>	<code>top_factors</code> for the predicted class, <code>predicted_label</code> , <code>class_probabilities</code> (all 3 classes)
<code>/predict/success-probability</code>	<code>top_factors</code> (3 features), <code>predicted_success</code> (bool), <code>decision_threshold</code>
<code>/predict/learning-growth</code>	<code>learning_speed</code> and <code>skill_growth</code> values together (regression — no SHAP; see Section 6)
<code>/predict/completion-probability</code>	derived_from note + the <code>dropout_risk</code> value it was computed from
<code>/predict/project-success-probability</code>	<code>top_factors</code> , plus a note that this approximates project-level success from the overall Success Probability model

5. Aggregate Summary Plots

In addition to per-prediction explanations, aggregate SHAP summary plots are generated for the three tree-based classifiers and saved to `app/ml/saved_models/`: `shap_summary.png` (Dropout Risk), `shap_summary_performance.png` (Performance Trend), and `shap_summary_success.png` (Success Probability). These give a global view of which features matter most across the whole test set, complementing the per-intern explanations returned by the API.

6. Explainability Scope by Model Type

SHAP explanations apply to the three tree-based classifiers (Dropout Risk, Performance Trend, Success Probability). The Learning Speed / Skill Growth model is a Linear Regression, whose coefficients are already directly interpretable by construction — each feature's effect on the prediction is a fixed, constant weight rather than something that needs to be estimated per-prediction, so a full SHAP pass was not required for this model to remain explainable.

7. Model Performance Context

Explanations are only as trustworthy as the model producing them. All four models' evaluation metrics are documented in full in the project's Model Evaluation Report; in summary: Dropout Risk (85.1% accuracy, ROC-AUC 0.842), Performance Trend (83.4%

accuracy across 3 balanced classes), Success Probability (90.5% accuracy, ROC-AUC 0.947), and Learning Speed / Skill Growth (R^2 of 0.594 and 0.023 respectively). These figures are reported honestly, including the comparatively low R^2 for Skill Growth, since a model whose accuracy looks artificially perfect is a stronger warning sign for hidden data leakage than a genuine explanation of true skill.

8. A Real Leakage Case, Caught Through Explainability Review

During development, an earlier version of the Performance Trend and Skill Growth models achieved a suspicious ~99% accuracy. Reviewing which features were driving predictions (the same kind of check this explainability layer performs on every live prediction) revealed the cause: several input columns were near-exact mathematical reconstructions of the label itself (for example, three task-status ratio columns that always summed to 1, one of which was the value the label was built from). The feature set was redesigned so each model must genuinely generalize from independent signals rather than reconstruct the answer from a disguised copy of it — documented in full in the Model Evaluation Report. This is offered here as concrete evidence that explainability was used as an active debugging and validation tool during development, not only as a reporting feature added at the end.